SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

1. Testing as an Engineering Activity

Introduction:

Software testing as an engineering activity is a systematic and planned process used to identify defects and improve software quality.

Systematic Testing Approach:

- Prepare test cases based on software requirements.
- Test valid, invalid, and boundary inputs.
- Compare actual results with expected results.
- Record defects and verify corrections.

Tools and Standards:

Selenium is used for automated testing, Jira for defect tracking, and JUnit for unit testing. Following defined testing standards ensures consistency and proper documentation.

Techniques:

Equivalence Partitioning, Boundary Value Analysis, Regression Testing, and Automation Testing can replace random testing.

Example:

In a banking application, test a transfer of ₹2,000 from an account containing ₹5,000. The expected balance is ₹3,000. Also test insufficient balance and duplicate transactions.

Conclusion:

Systematic testing improves defect detection, reduces failures, and increases software reliability.

2. Structured Testing Process in SDLC

Introduction:

A structured testing process identifies defects at different stages of the Software Development Life Cycle (SDLC).

Stages:

1. Requirement Analysis: Understand ATM withdrawal requirements.
2. Test Planning: Decide testing methods, resources, and schedule.

3. Test Case Design: Prepare cases for valid and invalid withdrawals.
4. Test Execution: Run test cases and compare actual and expected results.
5. Defect Reporting: Record defects and communicate them to developers.
6. Retesting and Regression Testing: Verify fixes and ensure existing functions still work.

Example:

For a withdrawal of ₹2,000 from an account with ₹5,000, the remaining balance should be ₹3,000. Test incorrect PINs, insufficient balance, and withdrawals above the permitted limit.

Conclusion:

Following a structured SDLC testing process reduces defects, improves consistency, and ensures reliable ATM operations.

3. Errors, Defects, and Failures

Introduction:

Errors, defects, and failures are related concepts used to understand problems in software.

Definitions:

- Error: A human mistake made during development.
- Defect: A flaw in the software caused by an error.
- Failure: When the software does not behave as expected during execution.

Relationship: Error → Defect → Failure.

Example:

A developer writes incorrect login validation logic. This creates a defect in the program. When a valid user cannot log in, the application experiences a failure.

Similarly, an incorrect transaction amount displayed to a customer may indicate a software defect.

Importance:

Confusing these terms can lead to unclear defect reports and delays in fixing problems. Reports should include steps to reproduce the issue, expected results, actual results, and screenshots when useful.

Conclusion:

Understanding these concepts improves defect reporting, root-cause analysis, and software quality.

4. Defect Clustering and Testing Principles

Introduction:

Defect clustering means that a large number of defects are often concentrated in a small number of software modules. Testing principles help testers focus on these areas.

Important Principles:

- Defect clustering: Give more attention to modules with a history of defects.
- Early testing: Identify defects early in development.
- Risk-based testing: Test critical functions first.
- Exhaustive testing is impossible: Prioritize important scenarios.

Test Case Prioritization:

Test cases should be ranked based on risk, severity, frequency of use, and previous defect history.

Example:

In a banking application, if most defects occur in fund transfers, testers should prioritize balance validation, transaction limits, and duplicate transaction checks.

Conclusion:

Applying defect clustering and testing principles improves test efficiency, detects important defects early, and reduces testing delays.

5. Defect Classification and Repository

Introduction:

A defect repository is a centralized location for storing defect details. Defect classification helps organize defects and prevent repeated problems.

Defect Classification:

Defects may be functional, performance-related, security-related, interface-related, or logical.

Repository Details:

A repository stores the defect ID, description, severity, priority, root cause, status, and resolution.

Example:

If a banking application deducts money twice during a transaction, the tester records the defect in the repository. The team analyzes the cause and creates additional test cases to prevent recurrence.

Prevention Methods:

- Maintain updated defect records.
- Analyze recurring defect patterns.

- Perform regression testing after corrections.
- Share lessons learned with developers and testers.

Conclusion:

Defect classification and repositories improve test design, reduce repeated defects, and enhance software quality.

6. Testing a Mobile Banking Application

Introduction:

Systematic software testing helps identify the causes of application crashes and improves reliability during fund transfers.

Testing Approach:

- Check coding errors, network problems, and memory issues.
- Perform functional testing of fund transfers.
- Test different network conditions and transaction amounts.
- Conduct stress testing under heavy usage.
- Perform regression testing after fixing defects.

Example:

If a mobile banking application crashes during a fund transfer, testers check whether the money was deducted and whether the transaction was completed only once. The defect is reported to developers for correction.

Improvements:

Use automated tests, proper defect tracking, and regular regression testing to prevent future failures.

Conclusion:

Systematic testing identifies root causes, reduces crashes, prevents transaction errors, and improves banking application reliability.

7. Testing vs Debugging

Introduction:

Testing and debugging are important software development activities, but they have different purposes.

Differences:

- Testing: Identifies defects by executing software and comparing actual results with expected results.

- Debugging: Finds the root cause of a defect and corrects the code.
- Testing is commonly performed by testers, while debugging is commonly performed by developers.
- Testing reports problems, whereas debugging investigates and fixes them.

Example:

If a login page rejects a valid password, the tester reports the failure. The developer debugs the code, identifies incorrect validation logic, and fixes it. The tester then retests the application.

Structured Approach:

Testers should report defects clearly, developers should debug them, and testers should verify fixes through retesting and regression testing.

Conclusion:

Testing and debugging work together to detect defects, correct errors, and improve software quality.

8. Cross-Browser Testing

Introduction:

Cross-browser testing checks whether a web application works correctly across different browsers and devices.

Testing Strategy:

- Test the application in Chrome, Firefox, and Edge.
- Check buttons, forms, layouts, and login functionality.
- Verify compatibility across different screen sizes.
- Record and fix browser-specific defects.

Tools: Selenium supports automated browser testing, while BrowserStack allows testing across different browsers and devices.

Example: An e-commerce website works correctly in Chrome, but the payment button does not respond in Firefox. Cross-browser testing helps identify and fix this compatibility defect.

Benefits: It improves compatibility, detects browser-specific problems, and provides a consistent user experience.

Conclusion: Systematic cross-browser testing improves application reliability and ensures that users can access the website through different browsers.

9. Defect Classification in a Payroll System

Introduction: Defect classification categorizes software problems according to their nature and helps testers identify and prevent errors.

Types of Defects:

- Functional defects: A feature does not work as required.
- Logical defects: Incorrect logic produces wrong results.
- Performance defects: The application responds slowly.
- Interface defects: Problems occur in the user interface.
- Data-related defects: Incorrect data is stored or processed.

Example: In a payroll system, an employee receives an incorrect salary because the tax calculation formula is wrong. This is a logical defect.

Prevention Methods: Test different salary ranges, validate tax formulas, apply Boundary Value Analysis, and perform regression testing after fixing defects.

Conclusion: Defect classification helps identify problem areas, improve test coverage, prevent recurring errors, and ensure accurate salary calculations.

10. Testing as an Engineering Activity

Introduction: Testing as an engineering activity replaces random testing with a planned and systematic process.

Structured Test Design:

- Understand the requirements.
- Prepare test cases with expected results.
- Test valid, invalid, and boundary values.
- Execute test cases and document defects.
- Verify corrections through retesting.

Testing Techniques: Equivalence Partitioning, Boundary Value Analysis, Regression Testing, and Automation Testing improve testing effectiveness.

Example: In a banking application, testers check withdrawals using valid amounts, insufficient balances, and amounts exceeding the withdrawal limit. The actual results are compared with expected results to identify defects.

Benefits: Structured test design improves test coverage, reduces missed defects, and makes testing more efficient.

Conclusion: A systematic engineering approach ensures better defect detection and more reliable software than random testing.

11. Data Inconsistency in a Healthcare Application

Introduction: Data inconsistency occurs when different modules display or store conflicting information. Testing helps identify these problems and maintain data integrity.

Causes of Defects:

- Incorrect coding
- Database update errors
- Improper data validation
- Communication problems between modules

Testing Approach:

- Perform integration testing between modules.
- Validate database records.
- Check data accuracy and consistency.
- Record defects and verify corrections.

Example: In a healthcare application, a patient's updated address appears in the registration module but not in the billing module. Integration testing helps identify the synchronization defect.

Improvements: Maintain proper defect reports, validate data updates, and perform regression testing after corrections.

Conclusion: Systematic testing ensures consistent patient information, improves data integrity, and increases application reliability.

12. Defect Repository for Repeated Defects

Introduction: A defect repository stores information about software defects and helps teams analyze and prevent recurring problems across releases.

Uses of a Defect Repository:

- Store defect descriptions and status.
- Record severity, priority, and root cause.
- Track corrections.
- Identify recurring defect patterns.

Example: If a login defect repeatedly occurs across software releases, testers review previous defect records and create additional test cases for login validation.

Prevention Methods:

- Update the repository regularly.
- Analyze defect patterns and root causes.
- Perform regression testing.
- Share previous defect information with developers and testers.

Conclusion: Effective use of a defect repository improves test case design, prevents repeated defects, and enhances software quality across releases.

13. Performance Testing of a Web-Based Learning System

Introduction: Performance testing checks whether a software application performs efficiently under different workloads.

Testing Gaps:

- Lack of load testing
- Insufficient test coverage
- Untested peak-usage scenarios

Testing Approach:

- Perform load testing and stress testing.
- Simulate multiple users accessing the system.
- Monitor response time and server performance.
- Identify and fix performance bottlenecks.

Example: During an online examination, hundreds of students access the learning system simultaneously, causing it to crash. Load testing helps identify this problem before deployment.

Improvements: Prepare realistic test scenarios, test peak workloads, and monitor system performance regularly.

Conclusion: Proper test planning and performance testing help prevent crashes, improve response time, and ensure system reliability during heavy usage.

14. Role of Tester in SDLC

Introduction: A tester plays an important role in the Software Development Life Cycle (SDLC) by

identifying defects and ensuring that software meets its requirements.

Role of the Tester:

- Review requirements and identify missing details.
- Prepare test plans and test cases.
- Execute tests and report defects.
- Verify corrections and perform regression testing.

Importance of Early Involvement: When testers participate during requirement analysis and design, they can identify problems early. This reduces correction costs and prevents defects from reaching later stages.

Example: In a banking application, a tester reviews fund transfer requirements and identifies missing validation for insufficient balance before coding begins.

Improvements: Involve testers from the beginning, encourage communication between developers and testers, and conduct regular review meetings.

Conclusion: Early tester involvement improves collaboration, detects defects sooner, reduces rework, and increases software quality.

15. Incomplete Test Coverage

Introduction: Test coverage measures how much of the software requirements, code, or functionality has been tested. Incomplete coverage may allow defects to remain undetected.

Causes:

- Missing test cases
- Untested inputs and conditions
- Ignoring boundary values
- Insufficient testing time

Improving Test Coverage:

- Prepare test cases for all requirements.
- Test valid, invalid, and boundary values.
- Use Equivalence Partitioning.
- Perform regression testing.
- Review test coverage to identify missing scenarios.

Example: In a banking application, withdrawals work for normal amounts but fail when the account balance is zero. Testing boundary conditions helps identify this defect.

Conclusion: Proper test coverage improves defect detection, validates software functionality, and increases application reliability.

16. Defect Reporting in a Logistics Application

Introduction: Defect reporting is the process of documenting software problems clearly so developers can understand, reproduce, and fix them.

Importance of Defect Reporting:

- Identifies defects accurately.
- Improves communication between testers and developers.
- Helps track defects until resolution.
- Reduces the time required to fix problems.

Elements of a Defect Report:

- Defect ID and title
- Steps to reproduce
- Expected and actual results
- Severity and priority
- Screenshots and status

Example: In a logistics application, the delivery status does not update after a package is delivered. The tester records the steps, expected status, actual status, and screenshot in a defect report.

Conclusion: Clear and structured defect reports improve communication, speed up defect resolution, and enhance software quality.

17. Process-Based Testing Approach

Introduction: Process-based testing follows a standard and systematic procedure to maintain consistency across different software modules.

Testing Approach:

- Prepare a standard test plan.
- Design test cases based on requirements.
- Follow common testing procedures.

- Record and report defects systematically.
- Perform regression testing after corrections.

Example: In a banking application, different teams test login and fund transfer using standardized test case formats and defect-reporting procedures. This ensures consistency across modules.

Benefits: Standardized processes reduce confusion, improve communication, prevent repeated defects, and make testing easier to manage.

Conclusion: A process-based testing approach improves consistency, detects defects effectively, and ensures reliable software across all modules.

18. Boundary Value Analysis

Introduction: Boundary Value Analysis (BVA) is a testing technique that checks values at the boundaries of an input range because defects frequently occur near these limits.

Why Edge Cases Are Missed:

- Incomplete test cases
- Insufficient test coverage
- Testing only normal inputs

Testing Approach: Test the minimum value, maximum value, and values immediately below and above the boundaries.

Example: If an application accepts ages from 18 to 60, test the values 17, 18, 19, 59, 60, and 61. This helps identify incorrect validation at the limits.

Improvements: Include edge cases in test plans, apply Boundary Value Analysis, and perform regression testing after fixing defects.

Conclusion: Boundary Value Analysis helps identify edge-case defects, improves test coverage, and increases software reliability.

19. Defect Repository and Documentation

Introduction: A defect repository stores details of software defects and helps teams track, analyze, and resolve problems effectively.

Importance of Documentation:

- Records defect details clearly.
- Tracks defect status and resolution.
- Helps identify recurring defects.

- Improves communication between testers and developers.

Repository Details: It contains the defect ID, description, severity, priority, root cause, status, and solution.

Example: In a banking application, a failed transaction is recorded with its error details and screenshots. Developers use this information to fix the problem, and testers verify the correction.

Improvements: Maintain updated defect records, use tools such as Jira, follow standard reporting procedures, and analyze previous defects to prevent recurrence.

Conclusion: Proper defect documentation improves tracking, simplifies analysis, prevents repeated defects, and enhances software quality.

20. Testing a Banking Transaction System

Introduction: Systematic software testing helps identify transaction defects and ensures the accuracy and reliability of banking operations.

Testing Approach:

- Analyze requirements and prepare a test plan.
- Design test cases for valid and invalid transactions.
- Execute tests and compare actual and expected results.
- Report defects and verify corrections.
- Perform regression testing after fixes.

Testing Techniques:

- Functional Testing
- Boundary Value Analysis
- Integration Testing
- Regression Testing
- Automation Testing

Example: During a fund transfer, a banking application deducts money twice. Testers reproduce the issue, report the defect, and verify the fix. They also check that each transaction is processed only once, even when network problems occur.

Improvements: Use automated regression tests, maintain defect records, and test failure and recovery scenarios.

Conclusion: Structured testing improves defect detection, prevents repeated transaction failures.

Code of Conduct:

*I **Sadhana K/ 192421137** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sadhana K

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand